



PRLD5121

PROJECT

Question 1

The first two steps

Understand the problem they use this problem to

- One of the most difficult aspects of programming

Users or end users

- People for whom a program is written

Documentation

- Supporting paperwork for a program

Plan the logic they use this problem to

An **algorithm** is the sequence of steps or rules you follow to solve a problem

Most common planning tools

Flowcharts

Pseudocode

IPO charts (input, processing, and output)

TOE charts (tasks, objects, and events)

Desk-checking

Walking through a program's logic on paper before you actually write the program

These phases are crucial because they enable the programmer to clearly comprehend the system's requirements prior to starting development. Rushing into coding without thoroughly analyzing the requirements can lead to the final system having calculation mistakes, flawed logic, or omitted features.

Q.1.2Technology and IT

The healthcare sector has faced catastrophic failures when systemic issues were overlooked. The **Mid-Staffordshire NHS crisis** in the UK resulted in hundreds of preventable deaths due to poor

care, misaligned priorities, and suppressed dissent within the organization, highlighting the consequences of failing to understand operational problems and patient needs.

leadershipstorybank.com

Q.1.3 maintain the program

maintenance ensure that the system remains accurate and useful over time

Maintenance

Making changes after the program is put into production

Common first programming job

Maintaining previously written programs

Make changes to existing programs

Repeat the development cycle

Question 2

Q.2.1 declaration of global parallel array

```
DECLARE intStudNum AS INTEGER ARRAY
```

```
intStudNum ← [101,102,103,104,105,106,107,108,109,110]
```

```
DECLARE strFirstName AS STRING ARRAY
```

```
strFirstName ←
```

```
[  
    "John","Jack","Jane","Jennifer","Jacqui",  
    "Jax","Jade","Joey","Joe","Jessie"  
]
```

```
DECLARE strSurname AS STRING ARRAY
```

```
strSurname ←
```

```
[  
    "Grey","Segal","Franks","Ophirton","Adam",  
    "Wexler","Gunther","Sparks","Helm","Morella"  
]
```

```
DECLARE strSubjCode AS STRING ARRAY
```

```
strSubjCode ←
```

```
[
```

```
"PROG1A",  
"INFOSS1",  
"DATABS1"  
]
```

```
DECLARE strSubjName AS STRING ARRAY
```

```
strSubjName ←
```

```
[  
  "Programming 1A",  
  "Information Systems 1",  
  "Databases 1"
```

```
]Q.2.2 // Results Arrays
```

```
DECLARE resultSubjCode AS STRING ARRAY[30]
```

```
DECLARE resultTest AS INTEGER ARRAY[30]
```

```
DECLARE resultAssignment1 AS INTEGER ARRAY[30]
```

```
DECLARE resultAssignment2 AS INTEGER ARRAY[30]
```

```
DECLARE resultStudNum AS INTEGER ARRAY[30]
```

Question 3

Q.3.1// input module

FOR index $\leftarrow$ 0 TO 29

INPUT resultSubjCode[index]

INPUT resultTest[index]

INPUT resultAssignment1[index]

INPUT resultAssignment2[index]

INPUT resultStudNum[index]

ENDFOR

END

Q.3.2// semester mark

FOR index $\leftarrow$ 0 TO 29

semesterMark[index] $\leftarrow$

```
(resultTest[index] × 0.40)
+
(resultAssignment1[index] × 0.25)
+
(resultAssignment2[index] × 0.35)
```

```
ENDFOR
```

```
END
```

```
// student who do not qualify
```

```
FOR index ← 0 TO 29
```

```
IF semesterMark[index] < 40 THEN
```

```
    OUTPUT resultStudNum[index]
```

```
    OUTPUT resultSubjCode[index]
```

```
ENDIF
```

```
ENDFOR
```

```
END
```

Q.3.3

```
// CalculateAverage
```

```
total  $\leftarrow$  0
```

```
count  $\leftarrow$  0
```

```
FOR index  $\leftarrow$  0 TO 29
```

```
IF resultSubjCode[index] = subjectCode THEN
```

```
total  $\leftarrow$  total + assessmentArray[index]
```

```
count  $\leftarrow$  count + 1
```

```
ENDIF
```

```
ENDFOR
```

```
RETURN total  $\div$  count
```

```
END
```

Q.3.4.1

```
// input ExamMark
```

```
FOR index  $\leftarrow$  0 TO 29
```


IF semesterMark[index] >= 40 THEN

INPUT examMark[index]

ELSE

examMark[index] $\leftarrow$ 0

OUTPUT "Student Does Not Qualify"

ENDIF

ENDFOR

END

3.4.2

// final mark

(

semester,

exam

)

RETURN

(semester $\times$ 0.50)

+

(exam × 0.50)

END

3.4.3

//FUNCTION GetAcademicStanding(finalResult)

3 IF finalResult >= 80 THEN

 RETURN "Distinction"

ELSE

 IF finalResult >= 60 THEN

 RETURN "Merit"

ELSE

 IF finalResult >= 50 THEN

 RETURN "Pass"

ELSE

 RETURN "Fail"

ENDIF

ENDIF

ENDIF

END

Q.3.5

//Subject Name and Display

INPUT searchSubject

found $\leftarrow$ FALSE

passed $\leftarrow$ 0

failed $\leftarrow$ 0

atRisk $\leftarrow$ 0

total $\leftarrow$ 0

FOR index $\leftarrow$ 0 TO 29

IF resultSubjCode[index]

=

searchSubject THEN

found $\leftarrow$ TRUE

total $\leftarrow$ total + 1

IF finalMark[index] $\geq$ 50 THEN

passed $\leftarrow$ passed + 1

ELSE

failed $\leftarrow$ failed + 1

ENDIF

IF finalMark[index] $<$ 30 THEN

atRisk $\leftarrow$ atRisk + 1

ENDIF

ENDIF

ENDFOR

IF found = TRUE THEN

riskPercentage

←

$(\text{atRisk} \div \text{total}) \times 100$

OUTPUT passed

OUTPUT failed

OUTPUT riskPercentage

ELSE

OUTPUT "Subject Not Found"

ENDIF

END

Q.3.6

// Reporting and Sorting

MODULE SortBySurname

FOR outer ← 0 TO 8

FOR inner ← 0 TO 8

IF strSurname[inner]

>

strSurname[inner + 1] THEN

CALL SwapStudents

(

inner,

inner + 1

)

ENDIF

ENDFOR

ENDFOR

END

3.6.2

// Display the sorted data

tempNum ← intStudNum[first]

intStudNum[first]

←

intStudNum[second]

intStudNum[second]

←

tempNum

tempFirst ← strFirstName[first]

strFirstName[first]

←

strFirstName[second]

strFirstName[second]

←

tempFirst

tempSurname ← strSurname[first]

strSurname[first]

←

strSurname[second]

strSurname[second]

←

tempSurname

END

3.6.3 bubble sort

It compares surnames next to each other and swaps them if they are in the wrong order.

- It is simple to understand
- Easy to implement using parallel arrays

Question 4



Q.4.1

Q.4.2 The system awards a 5% bonus to students who score above 70% in

- Test
- Assignment 1
- Assignment 2

Nested decisions are used so that every condition must be satisfied before the bonus is added.

```
IF resultTest[i] > 70 THEN
```

```
    IF resultAssignment1[i] > 70 THEN
```

```
        IF resultAssignment2[i] > 70 THEN
```

```
            finalMark[i]  $\leftarrow$  finalMark[i] + 5
```

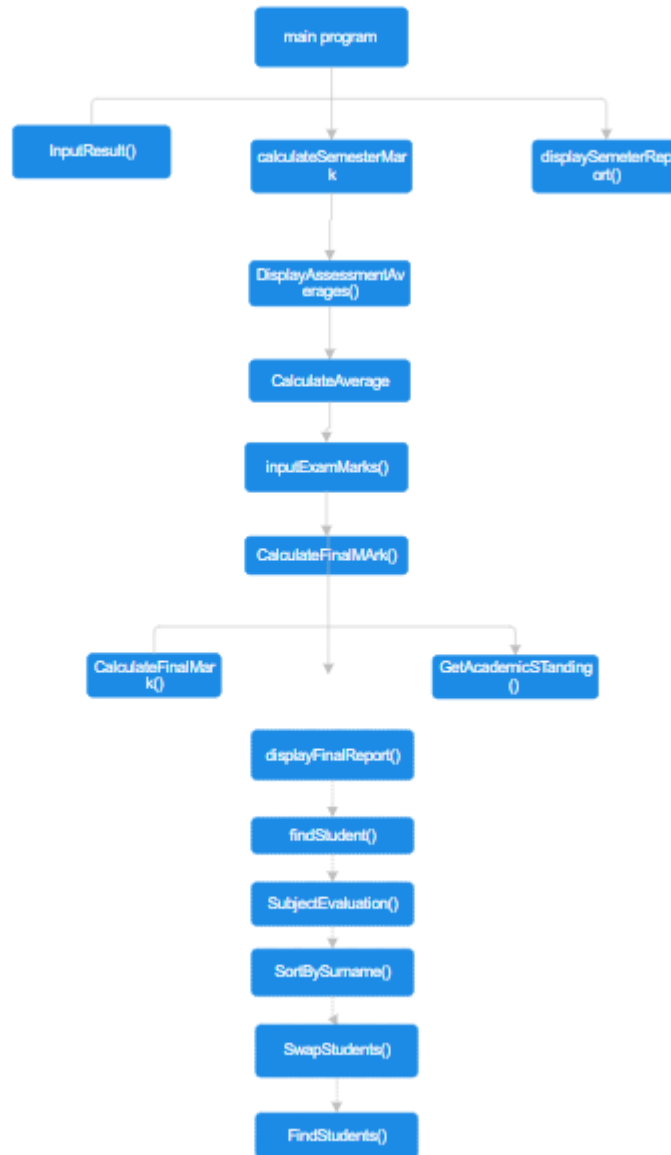
```
            receivedBonus[i]  $\leftarrow$  TRUE
```

```
        ENDIF
```

```
    ENDIF
```

```
ENDIF
```

The effect on report and sorting



Q.4.3

Q.4.4reflection

A significant challenge faced during development was handling parallel arrays. Because student information is distributed across several arrays, it was crucial to maintain proper alignment of all indexes. If any array became out of sync, incorrect student details or grades might be displayed in the reports

- Planning the structure carefully
- Using helper methods such as FindStudent
- Testing outputs after each module
- Applying modular programming
- Reusing functions to reduce duplicated logic

The hierarchy chart demonstrates how reusable modules simplified debugging and improved organisation.

Regular testing helped identify indexing problems early in development.

References

(farrell, programming logic and design , 2018)

Q.4.6 code documentation

- Purpose of modules
 - Function parameters
 - Return values
 - Validation rules
 - Processing logic

Documentation improves readability,
maintenance, debugging, and teamwork.